Assessment Brief (A3)

Assessment Details

Unit Code Title	PROG2002
Assessment #	Assessment 3
Assessment Type	Website (Complete dynamic website)
Due Date	12 October 2026 11:59 pm AEST/AEDT (Monday of Week 7)
Grades Release	Approximately 1 week after the submission date
Weight	40%
Length / Duration	N/A
Individual / Group	Group (2 students per team)
Unit Learning Outcomes (ULOS)	This assessment evaluates your achievement of the following Unit Learning Outcomes: ULOs 1-3 ULO1: understand understand and apply client-side and server-side web development technologies. ULO2: demonstrate the ability to work effectively and cooperatively in a web development team ULO3: plan, design, develop, and deploy a complete dynamic website.
GenAI Use Level	Level 2. Purpose-Specific GenAI Use Permitted See 'Academic Integrity' section below

Rationale

The purpose of this assessment is to test your ability to:

- ULO1: understand and apply client-side and server-side web development technologies.
- ULO2: demonstrate the ability to work effectively and cooperatively in a web development team
- ULO3: plan, design, develop, and deploy a complete dynamic website.

Your work will demonstrate your learning over the first four modules of this unit by **developing a complex dynamic website** using client-side and server-side web development technologies and **working in a team**.

Task Description

In this assessment, your task is to **develop a complete dynamic website** that involves **both client-side and admin-side websites**.

You will **use the same case study used in Assessment 2**. However, the complexity is further improved by adding more capabilities for the client-site website, including more APIs, and developing the admin-side website. Previously, you developed the client-side website, but it does not have the "register" feature yet. It also does not involve the server-side website, thus you are unable to manipulate the charity events (e.g., adding a new event, updating, or deleting events).

The assessment will also require students **to deploy both websites to the SCU cPanel** to demonstrate complete skills in developing a dynamic website. **Each student will deploy the applications to their own SCU cPanel**.

This assessment requires students to work in a team **(2 students)** in order to demonstrate the ability to work effectively and cooperatively in a web development team. *The team may decide how they will reuse the client-side websites from Assessment 2 (e.g., picking one of them or combining them)*.

Once your team is formed, **inform UA by sending an email and mentioning your student IDs, names, and cohort**. Students **must come from the same cohort** as the marking will be performed by your tutor. When the number of students are imbalanced, a group may have three students requiring UA approval. When this happens, the team may discuss further with the tutor to adjust the project requirements accordingly.

This assessment focuses on the following requirements:

1. **Improve the client-site website** by adding these features.
    a. A new page, named **the registration** page, is to allow web users to register for an event. Web users may decide how many tickets they want to purchase.
    b. Improve the current **event** page.
        - It displays the list of tickets purchased at the event, sorted by the latest date when the tickets were purchased. Previously, the page did not have this capability.
2. **Develop the admin-side website** to manipulate charity events and event category data.
3. **Add more relevant APIs** to serve the requirements above.
4. **Deploy both websites to the SCU cPanel** so they can be accessed publicly on the Internet.

This assessment **must retrieve data from the server** through APIs since it is a dynamic website, not a static website.

In this assessment, you MUST:

- **Use NodeJS, HTML, JavaScript, DOM, MySQL, and relevant topics** (as described in unit modules). Other than these, the submission is not acceptable.
- **Demonstrate your work progress** by regularly uploading your code to your GitHub repository.
- **Record a video** to explain your code and demonstrate your understanding.

This assessment **does not** require you to use AngularJS framework. However, you will **get extra points** (see marking criteria) **if you develop the websites using AngularJS framework properly for both client-side and admin-side websites**.

You may have **some additional points as well if you include an external API** (i.e.,Weather API from api.open-meteo.com) to improve your website. Note that this is **optional**. See External API section below.

Instructions

1. Database improvements

This part is to improve your existing database in order to accommodate the new capabilities related to event registration, an important feature for a dynamic charity event website. You will demonstrate your ability to plan and adapt backend systems using MySQL and Node.js (covered in Modules 2-3).

Requirements:

- Schema analysis and update: Based on the new requirement above, analyse your existing database to determine the most effective way to store this new information.
- Table creation: Design and create a new table that establishes a logical relationship with your existing event table. Your new table must be able to record which event a user registered for. **One user may register for multiple events and can only register for an event once.**
    - Consider the essential information this table needs to store. **At a minimum**, this should include a unique identifier, the user details, the date of registration, contact details, the number of tickets purchased, and the connection to the specific event.
    - Ensure you define primary and foreign keys to create relationships between your tables where necessary.
- Initial data: Populate the tables with a minimum of 10 registrations in any events to provide a realistic dataset for your website.
- Backend update: Update your Node.js file (event_db.js) to reflect the changes in your database schema.

To limit complexity, you are not required to have a USER table.

Note: For marking and deployment purposes, ensure you export your final database schema and data to a single SQL file. Follow the provided tutorial if needed: https://www.youtube.com/watch?v=lbrVhjnM5MQ.

2. Restful API improvement

This part aims to improve your current RESTful API (using Node.js and Express.js) to support the new features and data manipulations required. This task requires you to move beyond basic data retrieval and implement the full C.R.U.D. cycle (Create, Read, Update, Delete) for your resources (Modules 2-3)

- **Improved event details**: Update one of your existing endpoints for a single event (by ID) so that it efficiently retrieves not only the event details but also the complete list of associated registrations for that specific event. This will be featured on the website's **event detail** page on the client-side website.
- **New APIs for event data manipulation**: Design and implement new endpoints to allow the **admin-side website** (Part 4) to manipulate event data.
    - An endpoint to insert a new charity event into the database. This endpoint simulates the functionality an administrator would use to add new events to the platform.

- An endpoint that allows an administrator to update the details of an existing charity event based on its unique ID.
- An endpoint to remove an existing charity event. Note that your logic must uphold data integrity meaning **the event can only be deleted if it has not yet received any registrations**. If registrations exist, the deletion attempt should be blocked, and an appropriate error response should be returned to the user.

As you design these APIs, think carefully about the principles of RESTful design.

- Consider the resources you need and the security implications of your endpoints.
- Design API URLs to be intuitive and logical, reflecting the resources

Once you complete this part, you may use Postman (covered in Module 4) to test and verify that your API endpoints work correctly.

You are NOT required to create endpoints to manipulate category and registration, but you may do it if you want.

3. Client-side website enhancement

This part requires you to enhance your existing client-side website to integrate the new registration capability introduced in Part 2. This part can be completed by **using NodeJS, HTML, JavaScript, DOM, Promises**. You are not required to develop the website using AngularJS framework, but you may if you want to and extra points will be added (see marking criteria).

You may use your creativity to decide the page layout as long as it shows the required information properly. You are encouraged to use some styling to improve visualisation.

Event detail page

This page has been created in Assessment 2. You must enhance the existing Event Details page to display the new registration data and update the button. The event page should have these new requirements as follows:

- Use your updated API endpoint to dynamically retrieve and display a list of all recorded registrations for the specific event. The page should clearly present this information to the user.
- Update the existing "Register" button so that it now correctly links to the new Event Registration Page.
- **Must call your APIs**.

Registration page

This is a new page and has not been created in Assessment 2. Design and implement a new dedicated page to handle user registration for a selected event as follows:

- Create a user-friendly form that collects the necessary details from the user to submit a new registration record (e.g., the user details, the date of registration, contact details, the number of tickets purchased, following your database schema).
- Display the event information about the selected charity event on this page (using your APIs) so the user is clear about the event they are registering for.
- Implement a button that finalise the registration.
    - Validates the user input. If validation fails, provide an appropriate client-side alert.
    - Calls the necessary API endpoint created in Part 2 to insert the new registration record into the database.
    - Upon successful submission, display a confirmation message to the user and handle the following navigation.
- You may utilise appropriate methods (e.g., URL Query Strings or Local Storage) to pass the event ID between pages and fetch the correct data via your API.

4. Admin-side website development

This section requires you to develop a dedicated admin-side website for the staff. This site is the primary interface for manipulating the core event data using the full C.R.U.D. functionality you implemented in Part 2. This part demonstrates your ability to build a full-stack, data-management application.

Your task is to design an administrative interface that provides staff with full control over the event management. You have the flexibility to design this **as a single-page dashboard or split the functionality across multiple pages.**

To limit the complexity, assume that authentication is not required.

This part can be completed by **using HTML, JavaScript, DOM, Promises**. You are not required to develop the website using AngularJS framework, but you may if you want to and extra points will be added (see marking criteria).

You may use your creativity to decide the page layout as long as it shows the required information properly. You are encouraged to use some styling to improve visualisation.

The website should have the requirements as follows:

- **List feature**: Design and implement an intuitive interface that displays a complete list of all registered events, regardless of their status (Active, Past, Suspended). This data must be retrieved by calling the appropriate API endpoint. The list should include key details necessary for administrative purposes.
- **New feature**: Design and implement an intuitive form that allows staff to create and submit new charity events to the database. Ensure a "Save/Submit" button on this form calls the endpoint created in Part 2. Implement proper validation to ensure staff members fill in all the essential required fields before the data is sent to the API.
- **Update feature**: Design and implement an intuitive form that allows staff to select and update the details of an existing event. This interface should also retrieve and display any associated registration data for that specific event, providing the staff member with useful

information. Ensure a "Save" button calls the endpoint created in Part 2. Implement necessary validation before submission.

- **Delete feature**: Implement a "Delete" button or link next to each event in the main listing. Ensure this button triggers the call to the endpoint created in Part 2. Provide clear user feedback, especially if the deletion is blocked due to existing registrations.
- **Navigation:** Implement a clear and functional menu that allows users to navigate seamlessly to the other required pages. This menu must be present on all pages.

5. Deploying the websites to SCU cPanel

To meet the ULO3, **you are required to upload your codes to the SCU cPanel server** so the website will fully function and demonstrate its functionalities.

Each student will receive the SCU cPanel login to access your account (check your SCU email). There will be a video to guide you on how to upload your website to the server. **See 'Session Recording' (on the left menu) > Additional videos to support the assessment.**

Note: **Each student must deploy your app and APIs to your own SCU cPanel**. Students are not allowed to share your credentials with others as this will breach your security policies.

6. External Weather API (optional)

If you want more challenges, you may get additional points if you manage to utilise an external API from Open-Meteo API to further improve your websites and implemented properly. The API is free for public and without registration.

You may need to further update your event table since the API requires longitude and latitude of the event location to get the weather predictions.

API output example:
https://api.open-meteo.com/v1/forecast?latitude=-33.87&longitude=151.21&daily=weather_code,temperature_2m_max,temperature_2m_min&timezone=Australia%2FSydney

Then, you may can translate the output into more meaningful outputs:

Code	Description
0	Clear sky
1, 2, 2003	Mainly clear, partly cloudy, and overcast
45, 48	Fog and depositing rime fog
51, 53, 55	Drizzle: Light, moderate, and dense
61, 63, 65	Rain: Light, moderate, and heavy
80, 81, 82	Rain showers: Light, moderate, and violent
95	Thunderstorm: Slight or moderate

7. Deliverables:

Peer evaluation review: To evaluate teamwork and individual contribution, individual team members are required to provide the teamwork and engagement to the assessment by filling in a peer evaluation form individually and confidentially at the end of the assessment.

The form needs to be sent directly to your tutor (not UA) before the assessment is due. The email title should be "PROG2002_A3_[Team_ID]_[Member_name]".

For example, "PROG2002_A3_GC1_NehemiaSugianto". You can find this form under *Assessment 3 folder*.

GitHub repository: To demonstate your team work, you must **create a repository on GitHub** to store your project work with all files and documents. You **must show your work progress** in this assignment by regularly committing your project to the GitHub repository **as a team**. In each commit, you need to provide a clear explanation of what changes you have made in this commit. **Failing to show the correct work progress will cause the assignment to fail.**

Include your GitHub link in your submission. Ensure you protect your GitHub so that other students will not be able to see that. You may invite your marker to be a collaborator.

Include your GitHub in the submission link. Ensure it is accessible.

Demo video: Similar to Assessment 2, in your video (max of 15 minutes), demonstrate your understanding of the concepts and code development by addressing these key questions. The questions below will guide you on how to demonstrate the ULOs. **Focus on your individual contributions you worked in this project.**

1. **Database and API architecture (ULO3: Plan & design)**
   Explain and demonstrate your backend architecture, specifically how your database schema was designed to support the 'Charity event' case study and how your RESTful API endpoints efficiently retrieve the data required for your main website pages (Home, Search, and Details).

   You may show the database schema/SQL file, walk through the Node.js/Express file to show the API logic, and demonstrate testing a key API endpoint (e.g., the search filter) using a browser or Postman.

2. **Data flow and interaction between API and website (ULO1: Apply & ULO3: Develop)**

   Explain the data flow and process how your website interacts with the API, initially from initiating API requests until you receive the responses and display the outputs on the website.

   You may show the code to send requests to your APIs, show how you receive the server's response, and finally, show the code that renders the data into HTML on the website.

3. **Website functionality demo (ULO3: Complete dynamic website)**
   Walk us through the live demo for the home page, search page, and event page, specifically the search feature, where you perform data filtering and validation.

Upload your video to your SCU OneDrive and create a sharable link to it.

Note: There is project documentation required.

Note: Failing to show the proper work progress will fail the assignment . You may be required to attend an interview to explain your code and demonstrate your understanding.

Deliverables & Submissions

Submit the work to the provided submission link on the Blackboard learning site. You will submit all documents as specified in the deliverables section. Note that you don't need to submit the repository but the link to the GitHub repository.

- All your source code in 2 zip files (usernameA3-clientside.zip, usernameA3-adminside.zip and usernameA3-api.zip)
- The link to your GitHub repository.
- A video file/link.
- Peer evaluation form

Resources

To complete the task, you are recommended to:

- Study modules 1 - 4 materials and take an active role in the weekly tutorial and workshop. The workshop and tutorial activities will take you through the steps and tools to complate software configuration and procurement managent practices.

Formatting and style

N/A

Referencing

N/A

Generative Artificial Intelligence (GenAI) Guidelines

For this assessment, GenAI tools **Level 2** may be **used for specific assessment tasks or purposes as identified and scaffolded by the Unit Assessor**.

This assessment permits limited use of Generative AI tools (Level 2).

You **may** use GenAI to:

- Brainstorming ideas or ideating
- Checking grammar
- Paraphrasing, editing tone or clarity
- Formatting document
- Formatting references or citations
- Designing layout templates

You **may not** use GenAI to:

- Create your report
- Generate text and/or code and copy and paste it into your report/code

For further information regarding conditions of use, speak to the Unit Assessor and refer to Generative AI for Students.

GenAI Use Declaration

You **must include one of the following statements** in your final submission, depending on whether or not you used Generative AI (GenAI) tools for this assessment.

A. If you **DID** use GenAI tools, include this statement:

> *I acknowledge that I have used GenAI tools to complete this assessment. I used <**GenAI tool(s)**> to <**specific purpose(s) of using GenAI**> within the parameters outlined in the Assessment Brief and by the Unit Assessor.*

B. If you **DID NOT** use GenAI tools, include this statement:

> I acknowledge that I have not knowingly used GenAI to complete this assessment.

IMPORTANT: Misuse of GenAI or failure to acknowledge its use may breach academic integrity rules. The Unit Assessor may also ask you to describe or demonstrate which GenAI tools you used, how you used them, and how your use complied with the assessment guidelines. Be ready to discuss this if asked.

Rules relating to Assessment and Examination

For further information regarding; extensions, special consideration, late submissions, resubmissions, grades, appeals and academic integrity, please refer to: Rules Relating to Awards - Rule 3 - Coursework Awards - Student Assessment and Examinations and How to apply for Special Consideration.

Academic Integrity Declaration

By submitting this assessment, I declare that:

I have read and understood SCU's Academic Integrity policies and referencing guidelines. I am aware of the consequences of academic misconduct and confirm that this submission is my own original work, referenced appropriately, and has not been previously submitted. I authorise its reproduction for authentication purposes and understand the implications of a false declaration. I have adhered to guidelines regarding Generative AI.

Special Consideration

Please refer to the Special Consideration section of the Policy.
https://policies.scu.edu.au/document/view-current.php?id=140

Late Submissions & Penalties

Please refer to the Late Submission & Penalties section of the Policy.
https://policies.scu.edu.au/view.current.php?id=00255

Grades & Feedback

Assessments that have been submitted by the due date will receive an SCU grade. Grades and feedback will be posted to the 'Grades and Feedback' section on the Blackboard unit site. Please allow 7 days for marks to be published.

... continued on next page ...

Assessment Rubric

Criteria	High Distinction (85–100%)	Distinction (75–84%)	Credit (65–74%)	Pass (50–64%)	Fail (0-49%)
Develop a database to serve crowdfunding scenario by applying the client and server web technologies (Part 1) (ULOs 1,3) **5%**	Designs and develops the required database **effectively and efficiently without errors** to manage fundraiser data.	Designs and develops the required database **with some errors or oversights** in managing fundraiser data.	Designs and develops the required database with **significant errors or oversights** in managing fundraiser data.	Designs and develops the required database **with fundamental errors** or **lacks basic understanding** of managing fundraiser data.	**Fails to design and develop the required database with critical errors** in managing fundraiser data or **no implementation**.
Develop APIs to manage crowdfunding data by applying the client and server web technologies (Part 2) (ULOs 1,3) **10%**	Demonstrates **exceptional** understanding and application of client and server web technologies to serve data through APIs. Designs and develops APIs **effectively and efficiently without errors** to manage fundraiser data.	Demonstrates a **solid understanding** and application of client and server web technologies to serve data through APIs **with minor errors or omissions**. Designs and develops APIs **with some errors or oversights** in managing fundraiser data.	Demonstrates a **basic understanding** of client and server web technologies to serve data through APIs but **with notable gaps or errors**. Designs and develops APIs with **significant errors or oversights** in managing fundraiser data.	Demonstrates a **minimal understanding** of client and server web technologies to serve data through APIs with **substantial errors or misunderstandings**. Designs and develops APIs **with fundamental errors** or **lacks basic understanding** of managing fundraiser data.	**Fails to demonstrate** a basic understanding of client and server web technologies to serve data through APIs, with critical errors or complete lack of application. **Fails to design and develop APIs with critical errors** in managing fundraiser data or **no implementation**.
Design, develop, and deploy a	Demonstrates **exceptional** understanding and application of client and server web	Demonstrates a **solid understanding** and application of client and	Demonstrates a **basic understanding** of client and server web technologies but	Demonstrates a **minimal understanding** of client and server web technologies with **substantial**	**Fails to demonstrate** a basic understanding of client and server web

dynamic <u>client-side</u> website to serve crowdfunding scenario by applying the client and server web technologies (Part 3) (ULOs 1,3) **30%** **ULO4**	technologies to solve complex crowdfunding scenario. Designs, develops, and deploys a dynamic client-side website to serve home, search, and fundraiser requirements **effectively and efficiently without errors** **Extra points will be added if students develop the site using AngularJS framework properly and/or the external weather API*	server web technologies **with minor errors or omissions** in solving crowdfunding scenario. Designs, develops, and deploys a dynamic client-side website to serve home, search, and fundraiser requirements **with some errors or oversights** **Extra points will be added if students develop the site using AngularJS framework properly and/or the external weather API*	**with notable gaps or errors** in applying them to crowdfunding scenario. Designs, develops, and deploys a dynamic client-side website to serve home, search, and fundraiser requirements with **significant errors or oversights**.	**errors or misunderstandings** in applying them to crowdfunding scenario. Designs, develops, and deploys a dynamic client-side website to serve home, search, and fundraiser requirements **with fundamental errors** or **lacks basic understanding**	technologies, with critical errors or complete lack of application to crowdfunding scenario. **Fails to design, develop, and deploys** a dynamic client-side website **with critical errors**, or **do not meet** basic requirements, or **no implementation**.
Design, develop, and deploy a dynamic <u>admin-side</u> website to serve crowdfunding scenario by applying the client and server web technologies (Part 3) (ULOs 1,3) **30%**	Demonstrates **exceptional** understanding and application of client and server web technologies to solve complex crowdfunding scenario. Designs, develops, and deploys a dynamic admin-side website to serve home, search, and fundraiser requirements **effectively and efficiently without errors** **Extra points will be added if students develop the site using AngularJS framework properly*	Demonstrates a **solid understanding** and application of client and server web technologies **with minor errors or omissions** in solving crowdfunding scenario. Designs, develops, and deploys a dynamic admin - side website to serve home, search, and fundraiser requirements **with some errors or oversights** **Extra points will be added if students develop the site*	Demonstrates a **basic understanding** of client and server web technologies but **with notable gaps or errors** in applying them to crowdfunding scenario. Designs, develops, and deploys a dynamic admin-side website to serve home, search, and fundraiser requirements with **significant errors or oversights**.	Demonstrates a **minimal understanding** of client and server web technologies with **substantial errors or misunderstandings** in applying them to crowdfunding scenario. Designs, develops, and deploys a dynamic admin-side website to serve home, search, and fundraiser requirements **with fundamental errors** or **lacks basic understanding**	**Fails to demonstrate** a basic understanding of client and server web technologies, with critical errors or complete lack of application to crowdfunding scenario. **Fails to design, develop and deploy** a dynamic admin-side website **with critical errors**, or **do not meet** basic requirements, or **no implementation**.

		using AngularJS framework properly			
Concept understanding (Comment, GitHub, video) (ULOs 1,3) **10%**	Demonstrates a **profound understanding** of client-side and server-side web development technologies for the case study **through code comments, work progress in GitHub, and video**.	Demonstrates a **thorough understanding** of client-side and server-side web development technologies for the case study **through code comments, work progress in GitHub, and video.**	Demonstrates a **basic understanding** of client-side and server-side web development technologies for the case study **through code comments, work progress in GitHub, and video.**	Demonstrates a **minimal understanding** of client-side and server-side web development technologies for the case study **through code comments, work progress in GitHub, and video.**	**Fails to demonstrate** a basic understanding of client-side and server-side web development technologies for the case study through code comments, work progress in GitHub, and video.
Demonstrates Teamwork **(ULO 2)** **15%**	Consistently attends group meetings on time, contributes significantly and meaningfully to discussions, independently carries out tasks, and demonstrates a cooperative and supportive attitude, significantly contributing to the project's success.	Regularly attends group meetings on time, often contributes meaningfully to discussions, mostly works independently on tasks, exhibits a cooperative and supportive attitude, and extensively contributes to the project's success.	Mostly attends group meetings on time, contributes meaningfully to discussions, carries out tasks with some support, demonstrates a fairly cooperative and supportive attitude, and fairly contributes to the project's success.	Sometimes attends meetings on time, occasionally contributes meaningfully to discussions, requires support for tasks, occasionally demonstrates a cooperative attitude, and contributes fairly to the project's success.	Mostly absent from meetings, prioritises individual goals over team responsibilities, hinders group processes, causing others to take on their tasks, negatively impacting the project's progress.

Description of SCU Grades

High Distinction:

The student's performance, in addition to satisfying all of the basic learning requirements, demonstrates distinctive insight and ability in researching, analysing and applying relevant skills and concepts, and shows exceptional ability to synthesise, integrate and evaluate knowledge. The student's performance could be described as outstanding in relation to the learning requirements specified.

Distinction:

The student's performance, in addition to satisfying all of the basic learning requirements, demonstrates distinctive insight and ability in researching, analysing and applying relevant skills and concepts, and shows a well-developed ability to synthesise, integrate and evaluate knowledge. The student's performance could be described as distinguished in relation to the learning requirements specified.

Credit:

The student's performance, in addition to satisfying all of the basic learning requirements specified, demonstrates insight and ability in researching, analysing and applying relevant skills and concepts. The student's performance could be described as competent in relation to the learning requirements specified.

Pass:

The student's performance satisfies all of the basic learning requirements specified and provides a sound basis for proceeding to higher-level studies in the subject area. The student's performance could be described as satisfactory in relation to the learning requirements specified.

Fail:

The student's performance fails to satisfy the learning requirements specified.